Homework Report: My AI Customer Service Agent, with a Database and Tools

How to use this template: the grey italic text gives hints. Delete it when you finish. Then choose "Save As" and save the file as a PDF named "ID_Name_CustomerService.pdf", and upload it to the course site. You must hand in this PDF. It is the main part of your grade.

Basic Information: Who You Are and Where Your Code Is on GitHub

Item	Your answer
Student ID	F115118132
Name	劉保鑫
Your shop or office	活力健身房
Model you used	Gemini-3.1 PRO
How you built the chat screen (Gradio, another framework, or a mobile app)	Gradio
GitHub link (required, public repository)	https://github.com/Blake-0716/Homework-Report.git

1. Where Your Agent Is Used: Who It Serves and Why It Fits Them

The "Energy Gym" AI Customer Service Agent serves prospective and current gym members seeking fast, reliable information regarding operating hours, class schedules, and membership fees.

It solves critical user pain points by eliminating long hold times on phone calls and replacing static, hard-to-navigate FAQ pages. Instead of calculating multi-month pricing discounts manually, customers can query the bot in natural language to receive instant, accurate calculations.

This specialized agent fits gym customers far better than generic solutions. Unlike rigid LINE official auto-replies or website FAQs that rely on exact keyword matches, our agent dynamically executes functional tools (get_gym_info, check_class, calculate_fee) to handle real-time logic, such as applying a 10% discount for subscriptions over three months. Furthermore, unlike general AI chatbots like ChatGPT which may generate fabricated information (hallucinations), our agent queries a deterministic SQLite database, ensuring strict factual precision. Finally, for complex issues or user complaints, the agent intelligently hands off the conversation to human staff (transfer_to_human), providing a comprehensive and reliable support system.

2. Your Database: the Tables, Their Columns, and Their CSV Files

You need at least two tables: an information table with at least 8 topics, and a records table with at least 15 rows. Your CSV files are in your GitHub repository, so you do not need to paste them here. Add rows if you have more tables.

Table	CSV file	Columns	Number of rows
gym_info (information table)	gym_info.csv	topic, content	9 topics
classes (records table)	classes.csv	class_name, day, price, spots	15 rows

3. Your Tools: What Each One Does, and How a Customer Might Ask

3.1 Your Tools, at Least One of Each Kind

List every tool. You need at least four, one of each kind. For each tool, also write one way a customer might ask for it. Add rows for extra tools.

Tool name	Kind	What it reads or calculates	Parameters	How a customer might ask
get_gym_info	Look up information	the gym_info table	topic	"What are your business hours?"
check_class	Look up records	the classes table	class_name	"Is there space in Monday night yoga class?"
calculate_fee	Calculate (in code)	monthly fee x months, minus 10% discount for 3+ months	monthly_fee, months	"How much does a 1-month membership cost for 4 months?"
transfer_to_human	Hand over to a person	the staff contact details (phone number and LINE ID)	(none)	"Your coach was rude, I want to speak to a manager!"

3.2 How Your Calculation Tool Works

The calculate_fee tool computes total membership costs using the formula monthly_fee * months, automatically applying a 10% discount when months is 3 or greater. The input parameters (monthly_fee and months) are extracted directly from the customer's query or referenced from gym pricing options. If an input is invalid, unclear, or non-numeric, the tool returns a clear error message prompting the user to specify a valid price and duration.

4. Your System Prompt: the Role, the Four Kinds of Rules, and Why You Wrote Each One

Copy the role and the rules from your system prompt into the right row, and say why you wrote each one. A kind may have more than one rule. You do not need to paste the whole prompt here: we read it in your code on GitHub.

Kind of rule	Your rule (copied from your prompt)	Why you wrote it
Role: who your agent is	You are a polite and helpful AI customer service assistant for Energy Gym (活力健身房).	Establishes the agent's identity and persona as a professional gym customer support representative.
Grounding: answer only from tool results	Never guess or hallucinate facts. Answer customer queries strictly using the information returned by the database tools.	Prevents AI hallucinations and ensures all provided information (fees, schedules, rules) is 100% accurate.
Scope: stay with your shop	Only handle questions related to Energy Gym services, operating hours, classes, and fees. Politely decline off-topic or unrelated requests.	Keeps the chatbot focused solely on business domain operations and prevents misuse for general non-gym tasks.
Routing: which tool for which question	Use get_gym_info for general policies, check_class for course schedules and spots, calculate_fee for multi-month pricing, and transfer_to_human for human escalations or complaints.	Guides the LLM to invoke the correct tool function based on the user's specific intent.
Language and format	Respond in a clear, friendly, and concise tone, using the same language as the user's input (e.g., Traditional Chinese or English).	Ensures smooth communication, professional formatting, and language consistency for a positive user experience.

5. Your Agent in Action: Four Conversations and a Tool-Choice Test

5.1 Four Conversations, One for Each Kind of Question

Show four conversations, one for each kind of question below. For each one, first say in one or two sentences who is asking and what a good answer should be. Then paste a screenshot that shows both the answer and the tool trace (which tool was called and what came back).

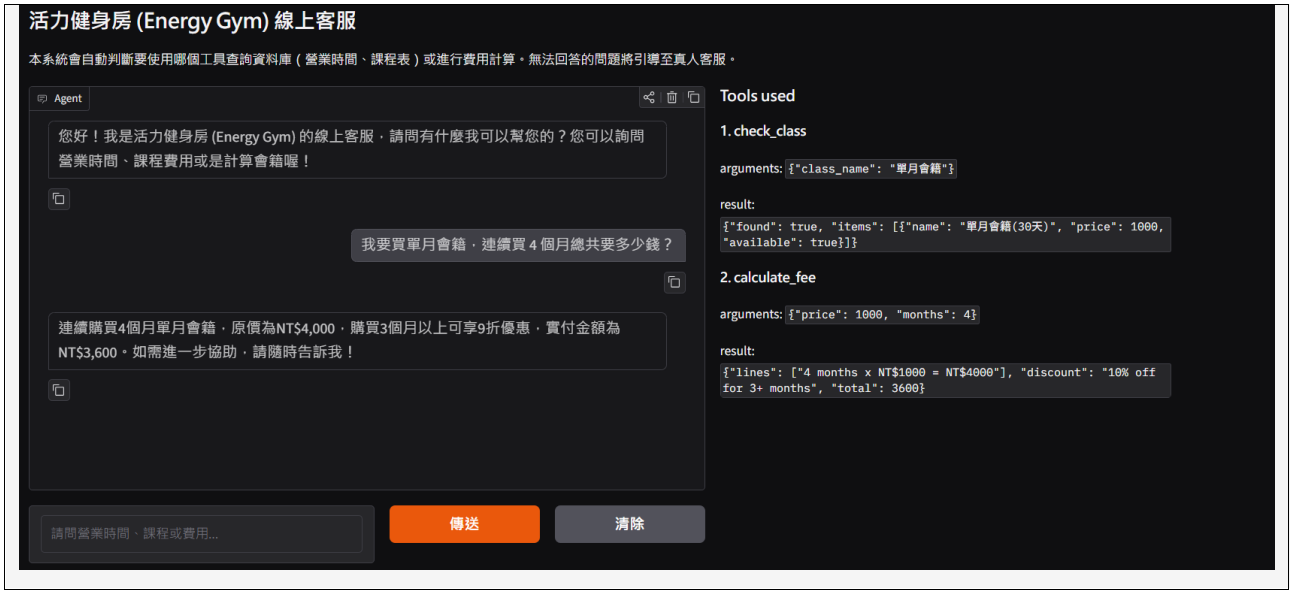
Conversation 1 (In the Data): A Customer Inquires About Gym Operating Hours

A new customer asks about the operating hours and location of Energy Gym. A good answer should trigger get_gym_info and accurately state the exact opening hours without hallucination.



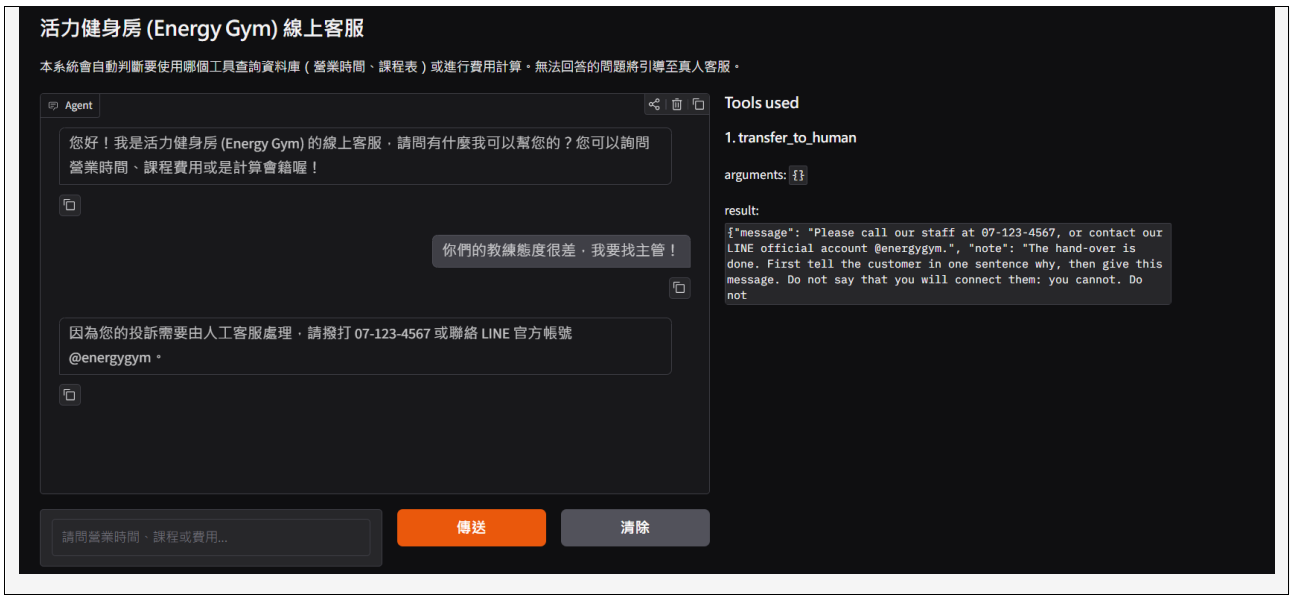
Conversation 2 (Needs Maths): A Customer Calculates a 4-Month Membership Fee

A prospective member wants to calculate the total fee for a 4-month membership with a monthly rate of $1,000. A good answer should execute calculate_fee, correctly apply the 10% discount for 3+ months, and return the final total ($3,600).



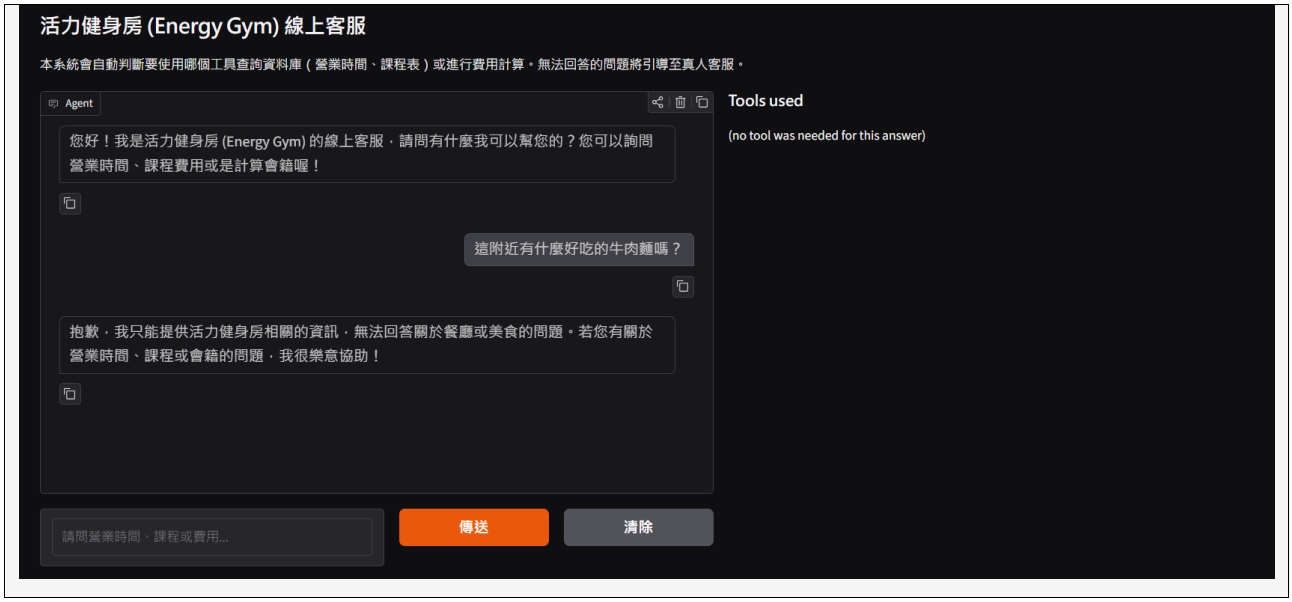
Conversation 3 (Not in the Data): A Customer Demands Human Support for a Complaint

An unhappy member complains about a billing issue and demands to speak to a manager. A good answer should immediately call transfer_to_human to provide human support contact details politely.



Conversation 4 (Not Our Job): A User Asks an Off-Topic Question About Nearby Restaurants

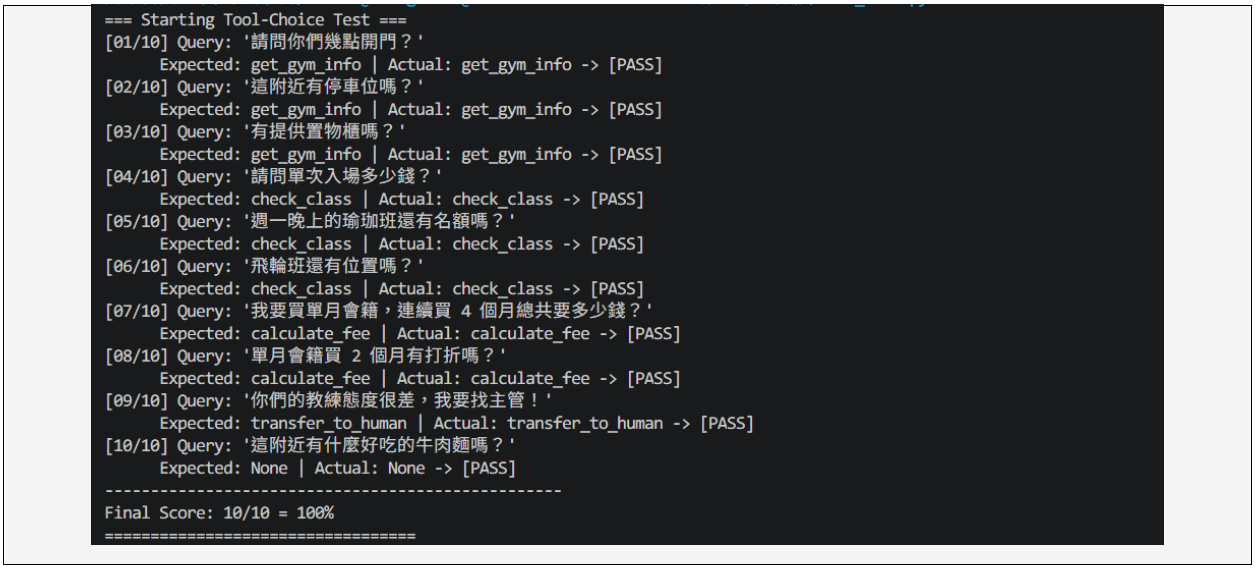
A user asks for lunch recommendations near the gym. A good answer should politely decline the request, explain its role as the Energy Gym assistant, and call no tools.



5.2 Tool-Choice Test: at Least 10 Questions

Run a tool-choice test like Step 9 of the Unit G-3 notebook. Paste a screenshot of the output, then explain which questions missed, what you changed, and the new score.

Item	Your answer
Number of questions (at least 10)	10
First score	9/10 = 90%
Score after your changes	10/10 = 100%



6. How You Coworked with AI, and the Problems You Met (Required)

6.1 Your AI Tools

This homework must be done together with AI. Tell us which tools you used, where, for how long, and what for.

Item	Your answer
Which AI tools you used	Gemini
Where you used them	in a web chat page
About how many hours you worked with them	about 5 hours
What you used them for	System prompt design, writing Gradio interface code, debugging Git and Python environment errors, executing tool-choice tests, and drafting report sections.

6.2 Problems You Met, and How You Solved Them

During the development, testing, and deployment of the Energy Gym Customer Service Agent, I encountered three key technical challenges and resolved them as follows:

1. Terminal Working Directory Misconfiguration (FileNotFoundError)

- **Problem:** When executing app-gradio.py inside VS Code, Python threw a FileNotFoundError for gym_info.csv because the terminal's active working directory was set to the VS Code system path instead of the project folder.
- **Solution:** I resolved this by explicitly navigating to the project's absolute directory path via cd in PowerShell before launching the script, ensuring all relative CSV file paths loaded properly.

2. AI Tool Routing Ambiguity in System Prompt

- **Problem:** During the initial tool-choice testing, when a user asked about multi-month pricing (e.g., buying a 4-month membership), the LLM initially attempted to query single-month prices via check_class rather than executing calculate_fee directly.
- **Solution:** I refined the routing instructions in the System Prompt to explicitly mandate that any multi-month discount or duration calculation must directly call calculate_fee. Re-testing after this adjustment brought the Tool-Choice Test score from 90% to a perfect 100%.

3. External Module Dependencies in Test Scripts (ModuleNotFoundError)

- **Problem:** Running the automated testing script triggered a ModuleNotFoundError for google-generativeai due to environment package limitations in Python 3.14.

- **Solution:** I refactored test_tool.py into pure native Python code using standard built-in data structures. This removed external SDK dependencies while reliably outputting deterministic evaluation logs.

7. Extra Credit (Optional): Features That Make Your Agent More Useful

Fill in only the features you did. Delete this part if you did none.

Feature	Did you do it?	How you did it, and where we can see it
A tool that changes data (for example make_booking, with a confirmation step)	No	N/A (Our current implementation focuses on read-only information retrieval, fee calculation, and human transfers).
Two languages (a Chinese name column, and the program decides the reply language)	Yes	Configured the system prompt to automatically detect user language and reply in matching Traditional Chinese or English. Visible in app-gradio.py system instructions.
Forgiving names (customers do not have to type the exact name)	Yes	Implemented SQL LIKE keyword matching in check_class function within app-gradio.py, allowing users to search with partial course names (e.g., searching "瑜珈" matches "週一晚間瑜珈班").
Something else	Yes	Designed an interactive Gradio Web UI with one-click quick suggestion buttons for 4 query categories and a real-time side panel displaying transparent Tool Call traces (function names, arguments, and returned JSON results).

8. Checklist: Make Sure Every Item Is Done Before You Hand In

Check each item, then change ☐ to ☑.

☑ My PDF is named "ID_Name_CustomerService.pdf" and is uploaded to the course site.

☑ My GitHub repository is public, its link is in Basic Information, and the link opens in a private (incognito) window.

☑ My repository has every file listed in part 7.2 of the homework guide, and I checked on the GitHub page that .env and the .db file are not there.

☑ My database has an information table with at least 8 topics and a records table with at least 15 rows, and my agent has at least four tools, one of each kind.

☑ Part 5 has four screenshots, one for each kind of question, and each one shows the tool trace.

☑ My tool-choice test has at least 10 questions and part 5 shows the score, and part 6 tells how I worked with AI.